
Exercise 1

1.a) Display default values of all primitive data types (Java)

Algorithm:

1. Start.
2. Define a class (e.g., DefaultValues) with instance variables for each primitive:
 - byte b;
 - short s;
 - int i;
 - long l;
 - float f;
 - double d;
 - char c;
 - boolean bool;(Note: Use instance variables so Java assigns default values automatically.)
3. Implement a method displayDefaults() that:
 - Prints a header: "Default Values of Primitive Data Types:"
 - Prints each variable name followed by its value (e.g., "Byte: " + b).
4. In main(), create an object of DefaultValues.
5. Call the displayDefaults() method on the object to print defaults.
6. End.

1.b) Roots of quadratic equation $ax^2 + bx + c = 0$ (compute discriminant D and classify roots)

Algorithm:

1. Start.
2. Read input coefficients a, b, c from the user.
3. Compute discriminant: $D = b^2 - 4ac$.
4. Print the value of D.
5. If $D > 0$:
 - Compute $\text{root1} = (-b + \sqrt{D}) / (2a)$.
 - Compute $\text{root2} = (-b - \sqrt{D}) / (2a)$.
 - Print "Roots are real and distinct." and both roots.
6. Else if $D == 0$:
 - Compute $\text{root} = -b / (2a)$.
 - Print "Roots are real and equal." and the single root.
7. Else ($D < 0$):
 - Compute $\text{realPart} = -b / (2a)$.
 - Compute $\text{imaginaryPart} = \sqrt{-D} / (2a)$.
 - Print "Roots are complex and imaginary." and both complex roots: $\text{realPart} \pm i \text{imaginaryPart}$.
8. End.

Exercise 2

2.a) Binary Search (on a sorted integer array)

Algorithm:

1. Start.
2. Read integer n (number of elements).
3. Read n sorted integers into array $\text{arr}[0..n-1]$.
4. Read the key value to search.
5. Initialize $\text{left} = 0$, $\text{right} = n - 1$.
6. While $\text{left} \leq \text{right}$:
 - a. $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$.
 - b. If $\text{arr}[\text{mid}] == \text{key}$:
 - Print index mid and exit (found).
 - c. Else if $\text{arr}[\text{mid}] > \text{key}$:
 - Set $\text{right} = \text{mid} - 1$.
 - d. Else:
 - Set $\text{left} = \text{mid} + 1$.

7. If loop ends without finding key:
 - Print "Element not found".
8. End.

2.b) Bubble Sort

Algorithm:

1. Start.
2. Read integer n.
3. Read n integers into array arr[0..n-1].
4. For i from 0 to n-2:
 - a. For j from 0 to n-i-2:
 - If arr[j] > arr[j+1], swap arr[j] and arr[j+1].
5. After sorting, print the sorted array.
6. End.

2.c) StringBuffer delete and deleteCharAt

Algorithm:

1. Start.
2. Read a string input from user; create a StringBuffer sb initialized with that string.
3. Print the original string (sb).
4. Read start and end indices for substring deletion.
5. Call sb.delete(start, end).
6. Print "String after deletion: " + sb.
7. Read index for deleting a character.
8. Call sb.deleteCharAt(index).
9. Print "String after removing character: " + sb.
10. End.

Exercise 3

3.a) Class mechanism: define class, methods, and call them

Algorithm:

1. Start.
2. Define a class Student with instance variables name (String) and age (int).
3. Implement method setDetails(String n, int a) to set name = n and age = a.
4. Implement method displayDetails() to print the name and age.
5. In main():
 - a. Create Student s1 = new Student().
 - b. Call s1.setDetails("Alice", 20).
 - c. Print header "Student Information:".
 - d. Call s1.displayDetails().
6. End.

3.b) Method overloading

Algorithm:

1. Start.
2. Define a class MathOperations containing multiple methods with the same name but different parameter lists, e.g.:
 - int add(int a, int b)
 - int add(int a, int b, int c)
 - double add(double a, double b)
3. In main():
 - a. Create instance of MathOperations.
 - b. Call each overloaded add method with appropriate parameters.
 - c. Print results.
4. End.

3.c) Constructor example

Algorithm:

1. Start.
2. Define a class Student with fields name and age.
3. Create a constructor Student(String n, int a) that sets name and age.
4. Implement display() to print name and age.

5. In main():
 - a. Create Student s1 = new Student("Alice", 20).
 - b. Call s1.display().
6. End.

3.d) Constructor overloading

Algorithm:

1. Start.
2. Define a class Student with fields name and age.
3. Provide multiple constructors:
 - Student() sets default values.
 - Student(String n) sets name and default age.
 - Student(String n, int a) sets name and age.
4. Implement display() to print name and age.
5. In main():
 - a. Create objects using each constructor: s1, s2, s3.
 - b. Print details for each using display().
6. End.

Exercise 4

4.a) Single Inheritance

Algorithm:

1. Start.
2. Define parent class Animal with method eat().
3. Define child class Dog extends Animal with method bark().
4. In main():
 - a. Create Dog myDog = new Dog().
 - b. Call myDog.eat() and myDog.bark().
5. End.

4.b) Multilevel Inheritance

Algorithm:

1. Start.
2. Define base class Animal with method eat().
3. Define class Mammal extends Animal with method walk().
4. Define class Dog extends Mammal with method bark().
5. In main():
 - a. Create Dog myDog = new Dog().
 - b. Call myDog.eat(), myDog.walk(), myDog.bark().
6. End.

4.c) Abstract class for areas of shapes

Algorithm:

1. Start.
2. Define abstract class Shape with abstract method calculateArea().
3. Define subclass Circle extends Shape:
 - a. Store radius.
 - b. Implement calculateArea() to compute and print $\pi * r * r$.
4. Define subclass Rectangle extends Shape:
 - a. Store length and width.
 - b. Implement calculateArea() to compute and print length * width.
5. Define subclass Triangle extends Shape:
 - a. Store base and height.
 - b. Implement calculateArea() to compute and print $0.5 * \text{base} * \text{height}$.
6. In main():
 - a. Create Shape references for each concrete shape with appropriate dimensions.
 - b. Call calculateArea() on each.
7. End.

Exercise 5

5.a) Example for `super` keyword

Algorithm:

1. Start.
2. Define parent class Animal with:
 - String name = "Animal"
 - Constructor Animal() prints a message.
 - Method makeSound().
3. Define child class Dog extends Animal with:
 - String name = "Dog"
 - Constructor Dog() calls super() then prints its own message.
 - Method display() that prints child's name, then prints super.name, then calls super.makeSound().
4. In main():
 - a. Create Dog myDog = new Dog().
 - b. Call myDog.display().
5. End.

5.b) Interface implementation and inheritance types

Algorithm:

1. Start.
2. Define interface Shape with method calculateArea().
3. Implement classes Circle and Rectangle that implement Shape and provide calculateArea().
4. In main():
 - a. Create Shape circle = new Circle(5), Shape rectangle = new Rectangle(4,6).
 - b. Call calculateArea() for both.
5. For multiple interface example:
 - a. Define interfaces Animal (eat()) and Bird (fly()).
 - b. Create class Bat implements Animal, Bird; implement both methods.
 - c. In main() create Bat and call both eat() and fly().
6. End.

5.c) Runtime polymorphism (method overriding)

Algorithm:

1. Start.
2. Define parent class Animal with method makeSound().
3. Define child classes Dog and Cat that override makeSound().
4. In main():
 - a. Declare Animal myAnimal.
 - b. Set myAnimal = new Dog(); call myAnimal.makeSound(); // Dog's method runs
 - c. Set myAnimal = new Cat(); call myAnimal.makeSound(); // Cat's method runs
5. End.

Exercise 6

6.a) Exception handling mechanism (single catch & finally)

Algorithm:

1. Start.
2. In main, wrap risky code in try block (e.g., integer division a / b).
3. Catch ArithmeticException and print an appropriate error message.
4. Use finally block to print "Execution completed." (executes regardless).
5. End.

6.b) Multiple catch clauses

Algorithm:

1. Start.
2. In try block, include code causing different exceptions, for example:
 - Division by zero (ArithmeticException)
 - Array indexing beyond limits (ArrayIndexOutOfBoundsException)
3. Provide separate catch blocks for ArithmeticException, ArrayIndexOutOfBoundsException, and a general Exception to catch any other exceptions.
4. In each catch, print a message explaining the error.
5. Use finally to print completion message.
6. End.

6.c) Demonstration of built-in exceptions

Algorithm:

1. Start.
2. Use separate try-catch blocks for each built-in exception:
 - a. `ArithmeticException` — divide by zero.
 - b. `ArrayIndexOutOfBoundsException` — access invalid index.
 - c. `NullPointerException` — call method on null reference.
3. Catch each exception and print the exception object or message.
4. After handling, continue program and print "Program execution continued..."
5. End.

6.d) User-defined exception

Algorithm:

1. Start.
2. Define a custom exception class `InvalidAgeException` that extends `Exception` and accepts a message in its constructor.
3. Implement a `validateAge(int age)` method that:
 - If `age < 18`, throw new `InvalidAgeException("Age must be 18 or above to vote.")`
 - Else print "You are eligible to vote."
4. In `main()`:
 - Call `validateAge` with a sample value (e.g., 16) inside try block.
 - Catch `InvalidAgeException` and print its message.
 - Print "Program execution continued..." after catch.
5. End.

Exercise 7

7.a) Creating threads (Extending Thread and using Runnable)

Algorithm (Extending Thread):

1. Start.
2. Define class `GoodMorningThread` extends `Thread`:
 - `run()`: loop forever (or for a criterion) { print "Good Morning"; `sleep(1000)`; }
3. Define class `HelloThread` extends `Thread`:
 - `run()`: loop { print "Hello"; `sleep(2000)`; }
4. Define class `WelcomeThread` extends `Thread`:
 - `run()`: loop { print "Welcome"; `sleep(3000)`; }
5. In `main()`:
 - Instantiate and start each thread: `new GoodMorningThread().start()`, etc.
6. End.

Algorithm (Using Runnable):

1. Start.
2. Define `GoodMorningRunnable` implements `Runnable`:
 - `run()`: loop { print "Good Morning"; `Thread.sleep(1000)`; }
3. Similarly define `HelloRunnable` (2s) and `WelcomeRunnable` (3s).
4. In `main()`: create `Thread t1 = new Thread(new GoodMorningRunnable()); t1.start()`; etc.
5. End.

7.b) `isAlive()` and `join()`

Algorithm:

1. Start.
2. Define a thread class `MyThread` extends `Thread` with `run()` printing its name and counting (sleep between iterations).
3. In `main()`:
 - a. Create instances `t1` and `t2`.
 - b. Start `t1` and `t2`.
 - c. Immediately print `t1.isAlive()` and `t2.isAlive()` (likely true).
 - d. Call `t1.join()` and `t2.join()` inside try/catch to wait for them to finish.
 - e. After joins, print `t1.isAlive()` and `t2.isAlive()` (should be false).
 - f. Print "Main thread execution completed."
4. End.

7.c) Daemon Threads

Algorithm:

1. Start.
2. Create a thread class (e.g., `DaemonThreadExample` extends `Thread`) whose `run` prints whether it's daemon or user thread and loops printing counts with sleep.
3. In `main()`:
 - a. Instantiate `t1 = new DaemonThreadExample()` (user thread).
 - b. Instantiate `t2 = new DaemonThreadExample()`; call `t2.setDaemon(true)` to mark as daemon.
 - c. Start both threads.
 - d. Let `main` sleep for a short time (e.g., 3000ms).
 - e. `Main` finishes — daemon thread terminates automatically when only daemon threads remain.
4. End.

7.d) Producer-Consumer Problem (shared buffer with wait/notify)

Algorithm:

1. Start.
2. Create a `SharedBuffer` class:
 - Maintain a `LinkedList` buffer and an `int` capacity.
 - `synchronized void produce(int item)`:
 - while `buffer.size() == capacity`: `wait()`;
 - add item to buffer; print "Produced: item"; `notify()`;
 - `synchronized int consume()`:
 - while `buffer.isEmpty()`: `wait()`;
 - remove and return first item; print "Consumed: item"; `notify()`;
3. Create `Producer` extends `Thread`:
 - `run()`: keep producing items by calling `buffer.produce(item)` and sleep to simulate delay.
4. Create `Consumer` extends `Thread`:
 - `run()`: keep consuming items by calling `buffer.consume()` and sleep to simulate delay.
5. In `main()`:
 - Create `SharedBuffer buffer = new SharedBuffer()`.
 - Create and start new `Producer(buffer)` and `Consumer(buffer)`.
6. End.

Exercise 8

8.a) Import and use user-defined packages

Algorithm:

1. Start.
2. Create a package directory (e.g., folder 'mypackage') and create `Calculator.java` inside with:
 - `package mypackage;`
 - `public class Calculator { public int add(int a,int b) {...} public int subtract... etc. }`
3. Compile the package files: `javac mypackage/Calculator.java`
4. In another file `MainProgram.java`:
 - `import mypackage.Calculator;`
 - In `main()`, instantiate `Calculator calc = new Calculator()`;
 - Use `calc.add()`, `calc.subtract()`, `calc.multiply()`, `calc.divide()` and print results.
5. Compile and run `MainProgram` class using `javac` and `java`.
6. End.

8.b) JavaFX GUI: Display text in Label and image in ImageView (design algorithm, no code required)

Algorithm:

1. Start.
2. Create a JavaFX Application subclass (extend `javafx.application.Application`).
3. In `start(Stage primaryStage)`:
 - a. Create a `Label` with text (e.g., "Hello, JavaFX!").
 - b. Load an `Image` (ensure the image file path is correct) and create an `ImageView` with that `Image`.
 - c. Optionally set `ImageView` properties: `fitWidth`, `preserveRatio`.
 - d. Place `Label` and `ImageView` into a layout container (`VBox` or `HBox`) and center them; add padding.
 - e. Create a `Scene` with the layout, set `Scene` on `primaryStage` and call `primaryStage.show()`.
4. End.

8.c) Tip Calculator app (JavaFX)

Algorithm:

1. Start.
2. Create JavaFX Application subclass.
3. In start():
 - a. Create Label and TextField for bill amount input (billInput).
 - b. Create a ChoiceBox (or ComboBox) for tip % (e.g., 10,15,20) and set a default.
 - c. Create a Button "Calculate Tip".
 - d. Create Labels to show tip amount and total amount.
 - e. On button click (setOnAction):
 - Try to parse billInput text as double; if parsing fails, show "Invalid input".
 - Get selected tip percentage.
 - Compute $\text{tipAmount} = \text{billAmount} * \text{tipPercentage} / 100$.
 - Compute $\text{totalAmount} = \text{billAmount} + \text{tipAmount}$.
 - Update tipAmountLabel and totalAmountLabel with formatted values.
 - f. Arrange nodes in VBox with padding and alignment, put Scene on Stage and show.
4. End.

Exercise 9

9.a) Implement programs using List interface and its classes (ArrayList, LinkedList, Vector, Stack)

General Algorithm (for each List-type class):

1. Start.
2. Create a collection instance (e.g., `ArrayList<String> fruits = new ArrayList<>();`).
3. Add elements using `add()`.
4. Print the entire list.
5. Perform a mutation (remove an element or `addFirst/addLast` for `LinkedList`).
6. Iterate over elements using for-each loop or for loop and print each element.
7. End.

Specifics:

- `LinkedList`: demonstrate `addFirst()` and `addLast()`, `removeFirst()`, `removeLast()`.
- `Vector`: show `capacity()`, `size()`, `add`, `remove`.
- `Stack`: push elements, pop one element, peek top element, check `isEmpty()`.

9.b) Implement programs using Set interface and its classes (HashSet, LinkedHashSet, TreeSet)

General Algorithm:

1. Start.
2. Create a set instance (e.g., `HashSet<String> colors = new HashSet<>();`).
3. Add elements, including duplicates to show duplicates are ignored.
4. Print the set (order not guaranteed for `HashSet`).
5. Remove an element and show `contains()` for membership test.
6. Iterate over set elements and print them.
7. End.

Specifics:

- `LinkedHashSet`: demonstrates insertion order preservation.
- `TreeSet`: demonstrates automatic sorting; show `first()`, `last()` and iteration order.

9.c) Implement programs using Map interface and its classes (HashMap, LinkedHashMap, TreeMap)

General Algorithm:

1. Start.
2. Create a map instance (e.g., `HashMap<Integer,String> students = new HashMap<>();`).
3. Put key-value pairs using `put(key, value)`.
4. Print the whole map.
5. Access a value using `get(key)`.
6. Remove an entry using `remove(key)`.
7. Iterate over map entries using `entrySet()` and print each key and value.
8. End.

Specifics:

- `LinkedHashMap`: demonstrates predictable (insertion) order.
- `TreeMap`: demonstrates keys sorted order; show `firstEntry()` and `lastEntry()`.

Notes

- Each algorithm above is written as simple step-by-step instructions to help implement the Java program.
 - For UI/JavaFX exercises, the algorithm focuses on component creation, arranging layout, and event handling.
 - For multi-threading and synchronization, algorithms emphasize thread creation, shared buffer locking, and wait/notify semantics.
-

End of Algorithms.